

POLITECNICO
MILANO 1863

Best Bike Paths, Luca Suzani, Alberto
Spezzapria, Alessio Pesavento

Acceptance Test Deliverable

Deliverable:	ATD
Title:	Acceptance Test Deliverable
Authors:	Luca Suzani, Alberto Spezzapria, Alessio Pesavento
Version:	1.0
Date:	08-February-2026
Download page:	https://github.com/Alessio01103/PesaventoSpezzapriaSuzani.git
Copyright:	Copyright © 2026, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

Contents

Table of Contents . . . 3
List of Figures . . . 4
List of Tables . . . 4

1 Introduction and Project Identification **6**
1.1 Purpose . . . 6
1.2 Reviwed Project Identification . . . 6
1.3 Source Code Repository . . . 6
1.4 Reference Documents . . . 6
1.5 Testing Team . . . 7

2 Installation Setup **8**
2.1 Installation Procedure . . . 8
2.2 Documentation Strengths . . . 8
2.3 Critical Issues and Problems Encountered . . . 9

3 Acceptance Test Cases and Results **10**
3.1 Manual Path Creation & Constraints . . . 10
3.2 Path Search Logic & Ranking Transparency . . . 10
3.3 History & Statistics Consistency . . . 11
3.4 Data Persistence Anomaly . . . 11
3.5 Summary of Issues . . . 11

4 Documentation and Code Quality **12**
4.1 RASD (Requirement Analysis and Specification Document) . . . 12
4.2 DD (Design Document) . . . 12
4.3 ITD (Implementation and Test Document) . . . 13
4.4 Presentation and Editorial Quality . . . 13

5 Effort Spent **15**

List of Figures

List of Tables

3.1 Summary of Acceptance Test Results 11

5.1 Hours of work spent by each author on installation, acceptance testing and document preparation 15

Chapter 1

Introduction and Project Identification

1.1 Purpose

This document reports the results of the Acceptance Testing phase performed on the **BestBikePaths (BBP)** application. The goal is to validate the software against the requirements specified in the RASD, verify the consistency of the architectural design, and assess the reproducibility of the installation process.

1.2 Reviwed Project Identification

The software project analyzed in this document is **BestBikePaths**. The application was designed and developed by the following team:

- **Andrea Fidanza** (Mat. 10730617)
- **Guglielmo Gatti** (Mat. 10794407)
- **Marco Maiocchi** (Mat. 10723893)

1.3 Source Code Repository

The testing procedures were performed using the source code, database dumps, and binary files hosted at the following public GitHub repository:

`https://github.com/topox19547/FidanzaGattiMaiocchi`

Specific focus was placed on the `/Implementation` folder for the source code and the `/DeliveryFolder` for the documentation and executable binaries.

1.4 Reference Documents

The evaluation was conducted based on the following official documents provided by the development team, which served as the "oracle" for expected system behavior:

1. **RASD** (Requirement Analysis and Specification Document) - Version 1.0
2. **DD** (Design Document) - Version 1.0
3. **ITD** (Implementation and Test Document) - Version 2.0

1.5 Testing Team

This Acceptance Test Document (ATD) has been produced by:

- **Alessio Pesavento**
- **Alberto Spezzapria**
- **Luca Suzani**

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

Chapter 2

Installation Setup

This section describes the activities performed to install the *Best Bike Paths* (BBP) system prototype and analyzes the overall experience, highlighting both the effectiveness of the instructions provided in the ITD document and the technical issues and gaps encountered.

2.1 Installation Procedure

The installation was executed by faithfully following the instructions reported in Chapter 6 of the ITD document. The environment was configured on a single machine hosting both the server and the client (via emulator), as suggested by the prerequisites. The procedure followed the indicated logical order:

1. Configuration of the PostgreSQL database and the PostGIS extension.
2. Import of the schema and data via the `DatabaseDump.sql` file.
3. Startup of the Server component using the provided `.jar` executable.
4. Installation of the Android Client via drag-and-drop of the `.apk` file into the emulator.

2.2 Documentation Strengths

The analysis of the installation guide highlighted several positive aspects that facilitated the initial startup:

- **Availability of Executables:** The provision of pre-compiled files `BestBikePathsServer.jar` and `BestBikePathsClient.apk` was fundamental. This allowed testing the system without necessarily having to recompile the entire source code, reducing potential build errors in the first instance.
- **Logical Sequence:** The structure of Chapter 6 is well-organized, guiding the user through the application layers in the correct order (Database $\rightarrow$ Server $\rightarrow$ Client), which is essential to avoid connection errors during the first startup.
- **Explicit Commands:** The commands provided for importing the database and starting the server (including flags to overwrite the default username and password) proved to be syntactically correct and functional.

2.3 Critical Issues and Problems Encountered

Despite the logical structure, the installation phase proved to be significantly time-consuming and complex due to various incompatibilities and informational gaps:

- **Software Compatibility Issues:** The use of different or older versions created conflicts that required time to resolve, forcing a non-trivial update or specific configuration of the environment.
- **Missing Configuration Details:** Although the main commands are present, the guide tends to take prior knowledge of specific tools for granted, creating difficulties for users not expert with certain software:
  - **Database Management:** The instructions for importing the SQL dump (`\i 'path/to/file'`) lack details regarding potential file read permission issues or path handling on different operating systems, which often generate silent errors.
  - **External API Configuration:** For those wishing to compile the client, the instructions on API key management are too brief. It is indicated to insert them into the `local.properties` file, but no guide is provided on how to generate these keys or which specific SDKs to enable in the respective developer consoles, risking having an installed app with non-functional maps.
- **Overall Setup Time:** Due to the aforementioned issues, the actual time required to obtain a functioning system was much higher than expected, turning what should have been a simple prototype installation into an execution environment debugging process.

In conclusion, the installation process proved to be **notably long and burdensome**. Although the documentation covers the necessary logical steps, the practical experience highlighted several points of friction due to software incompatibilities and a level of detail not always adequate for those approaching the ecosystem from scratch.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

Chapter 3

Acceptance Test Cases and Results

This chapter illustrates the results of the system validation phase. The tests were conducted by simulating real-world usage scenarios to verify functional compliance with initial requirements, interface usability, and the consistency of business logic.

The following sections detail the discrepancies found between the *Expected Behavior* and the *Actual Behavior*.

3.1 Manual Path Creation & Constraints

Test Objective: To evaluate the efficiency and ergonomics of the manual path creation process and the insertion of crowd-sourced data.

- **Design Constraint & Usability (150m Limit):** The system imposes a design constraint that limits the maximum distance between two consecutive nodes to 150 meters. While logically functional, this design choice significantly degrades the user experience (*UX Friction*), forcing the operator to perform a high number of interactions to cover real geographical distances. The operation results in being slow and unintuitive.
- **Input Handling (Touch Interaction):** An anomaly was detected in tactile interaction: selecting a point in the immediate vicinity of the last inserted node is interpreted by the system as a command to delete the existing node rather than as a new insertion or a null touch. This behavior increases the risk of accidental errors during mapping.
- **Status Reporting:** The functionality for reporting road conditions (*Status*) is correctly implemented, allowing the enrichment of the path with safety attributes.

3.2 Path Search Logic & Ranking Transparency

Test Objective: To verify the visualization of search results and the transparency of the ranking algorithm ("Best Path").

- **System Opacity:** The system proposes a "Best Path" and selectable alternatives to the user but provides no visibility regarding the calculated numerical score (*Score*) or the complete ranking of the analyzed paths. The user has no access to data justifying the selection of one path over another.
- **Inference on Ranking Logic:** In the absence of visible scores, empirical observation of the results suggests that the algorithm prioritizes efficiency metrics (shortest time/distance) at the expense of safety. It was observed that the system tends to recommend short paths as the "Best Path" even in the presence of danger warnings ("Dangerous"), while safer but longer paths are penalized or

relegated to secondary alternatives. This behavior appears to contradict the project's *SafetyFirst* philosophy.

- **Missing Preview:** It is not possible to view the route on the map before starting the activity, limiting the user's ability to visually evaluate the proposed path.

3.3 History & Statistics Consistency

Test Objective: To verify the persistence of historical data and the availability of post-activity metrics.

- **Absence of Statistics:** The *History* section is functionally limited regarding data analysis expectations. The system does not display essential metrics such as duration, average speed, or detailed timestamps of past activities.
- **Functional Limitation:** The only allowed interaction with a history item is restarting a new activity on the same path, without the possibility of consulting previous performance details.

3.4 Data Persistence Anomaly

Test Objective: To verify the correctness of the data lifecycle (Save/Discard activity).

- **Logic Error in Saving:** A functional defect was identified at the end of an activity. When the system requests confirmation to save data, the user's negative input (selecting "No") is ignored: the activity is archived in the *History* regardless. This bug compromises local database consistency and violates the user's explicit choice.

3.5 Summary of Issues

The following table summarizes the critical issues identified during the acceptance testing phase.

Functional Area	Issue Type	Description	Severity
Path Creation	Usability / Design	150m constraint is overly limiting; Accidental point deletion error.	Medium
Ranking	Transparency / Logic	Hidden Scores; Ranking appears to be based on duration rather than safety.	High
History	Missing Feature	Performance statistics are not viewable.	Medium
Persistence	Functional Bug	Activity is saved even if the user declines.	Low

Table 3.1: Summary of Acceptance Test Results

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

Chapter 4

Documentation and Code Quality

This section provides an evaluation of the delivered documentation. It analyzes the consistency between the initial specifications, the architectural choices and the final software product, highlighting both the theoretical work and the practical gaps encountered during the testing phase.

4.1 RASD (Requirement Analysis and Specification Document)

Strengths

- **Formalization and Safety Logic (Alloy):** The inclusion of a complete Alloy model is a point of excellence. Specifically, the **SafetyFirst** assertion (Image 4.2) formally defines the ranking logic, ensuring that dangerous paths are pessimistically penalized. This demonstrates a strong commitment to verifying the domain consistency before implementation.
- **Comprehensive Modeling:** The document provides extensive UML modeling, including Class Diagrams and State Diagrams, which effectively clarify the domain entities and their lifecycle.
- **Detailed Scenarios:** The description of usage scenarios (e.g., *Scenario 3: Manual Tracking*) is narrative and effective, helping the reader understand the expected logical flow "on paper."

Weaknesses

- **Missing Non-Functional Constraints (UX):** The document fails to specify critical usability constraints. The **150-meter limit** for point creation—which acts as a blocking factor in the software—is not mentioned in the specific requirements nor in the system limitations. This creates a discrepancy between the expectation created by the RASD (freedom of tracking) and the reality of the software.
- **Ambiguity of "Crowd-sourcing":** The RASD places emphasis on the social nature of the app. However, it does not address the risk of user base fragmentation resulting from the choice of a single platform (Android). This decision is taken for granted without an impact analysis on the core requirement of "massive data collection."

4.2 DD (Design Document)

Strengths

- **Architectural Layering:** The high-level architecture is clearly defined. The document effectively visualizes the separation of concerns through a 3-tier structure (Client, Server, Database), and the decomposition into modules (e.g., *Path Scoring*, *Trip Management*) is coherent with the functional requirements.

- **Integration Strategy:** The document outlines a progressive integration plan (described in steps, e.g., Step 9 to Step 11), showing a logical approach to assembling the backend and frontend components.

Weaknesses

- **Unjustified Over-Engineering:** The choice of the technology stack (Spring Boot + PostgreSQL/-PostGIS) is presented as a given, without a substantial *Rationale*. For a system managing simple geospatial entities, the absence of a discussion on lighter alternatives is an architectural oversight, leading to high development friction.
- **UI/UX Discrepancy:** The mockups and interface descriptions in the DD do not anticipate the complexity of map management (e.g., obscured play buttons, involuntary point deletion), suggesting an interface design that underwent little iteration.

4.3 ITD (Implementation and Test Document)

Strengths

- **Intellectual Honesty:** Section *2.3 Excluded Functionalities* clearly lists what has not been implemented, allowing the reviewer to avoid searching for non-existent features.
- **Adherence to Code Standards:** The described source code structure (MVC pattern on Spring, standard Android structure) appears to follow the best practices of the respective frameworks, despite the complexity.

Weaknesses

- **Non-Standardized Deployment:** In a modern DevOps context, requiring manual DB installation, specific user creation, and manual SQL script injection is considered an anti-pattern. The absence of a `docker-compose` file forces the reviewer to handle compatibility issues (PostGIS, Java versions) manually. Details on `Installation Setup` section.
- **Testing Friction:** No data seeding scripts were provided. Delivering an empty database for an app that derives its value from data (paths, ranking) prevents immediate testing and forces the reviewer to manually handle data entry.

4.4 Presentation and Editorial Quality

Although they do not directly impact software functionality, we detected some graphical and editorial inconsistencies that reduce the perceived quality and navigability of the documentats.

- **Lack of Uniformity:** There is no common visual identity among the documents. The RASD was evidently generated via Notion export, resulting in a style disconnected from the DD and ITD, which were drafted in LaTeX. This fragmentation leads to the absence of a common header/footer and inconsistent formatting styles, giving the impression of "disjointed" documents rather than a unified project corpus.
- **Absence of Dynamic Navigation:** Despite the use of LaTeXfor the DD and ITD, the generated PDFs are "static." Essential features for digital consultation, such as the bookmarks sidebar and clickable internal cross-references, are missing. In technical documents spanning dozens of pages, the lack of these tools makes navigation slow and cumbersome.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

- **Attention to Detail (Typos & Formatting):** Several minor editorial inaccuracies were found, such as inconsistent use of bold text to highlight key terms, capitalization errors, and non-uniform caption formatting. While minor, these details contribute to a less polished and professional overall appearance.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

Chapter 5

Effort Spent

Task	Date	Alessio P.	Alberto S.	Luca S.
Installation & Testing	04/02/2026	1h	1h	1h
	05/02/2026	2.5h	2h	1.5h
	06/02/2026	1h	1h	1.5h
	Subtotal	**4.5h**	**4h**	**4h**
Document Preparation	06/02/2026	1h	1h	1h
	08/02/2026	2h	3h	2h
	Subtotal	**3h**	**4h**	**3h**
TOTAL EFFORT		**7.5h**	**8h**	**7h**

Table 5.1: Hours of work spent by each author on installation, acceptance testing and document preparation